ECDR

Week 1: Project Brief and Research Grounding

ECDR Research

2 November 2026

Contents

1 Executive brief **2**

1.1 Problem and decision contract 2
1.2 Data and label contract 2
1.3 Frozen reference architecture 3
1.4 Research questions and hypotheses 3
1.5 Domain, event, and certificate semantics 4
1.6 Reference classes and architecture audit 4

2 Feasibility baseline results **4**

2.1 Objectives, split subjects, and pipeline 6
2.2 Certificate lifecycle and architecture-specific firewall 8
2.3 Grounding and research gap 8
2.4 Evidence status and roadmap 8

3 Source and reproducibility contract **9**

1 Executive brief

This Week 1 document fixes the research question, reference contract, evidence boundaries, and experimental plan for Event-Conditioned Certified Decision Reuse (ECDR) on the HAPT smartphone inertial-sensing data. The primary reference is a new, frozen twelve-class SmallCNN1D specification. The historical six-class checkpoint and the earlier valid-convolution/GAP theory remain provenance only. The completed feasibility baseline is reported below; it does not establish ECDR certification or net savings.

1.1 Problem and decision contract

Let $X_t \in \mathbb{R}^{6\times 128}$ be a raw sensor window and let P be fixed per-channel z-score preprocessing, fitted on training subjects only. The reference function is $F = f_\theta \circ P$, with logits $F(X) \in \mathbb{R}^{12}$ and deterministic tie-breaking. The reference decision is $h(X) = \arg\max_k F_k(X)$. An ECDR policy may reuse a previous decision only when its certificate proves that the current input belongs to a region on which the same fixed reference decision holds. The primary safety target is therefore $\hat{h}_{\text{ECDR}}(X) = h(X)$ with zero disagreements over the declared evaluation scope. Agreement with the reference is distinct from agreement with ground-truth labels.

The main research question is: for the frozen SmallCNN1D K=12 reference on HAPT, can an event-conditioned certificate preserve every reference decision while reducing total end-to-end inference cost under a declared streaming workload? Cost includes reference forward evaluation, preprocessing, event detection, membership tests, certificate construction or reuse, fallback evaluation, and I/O or scheduling overhead. Forward-only timing cannot establish a net saving.

1.2 Data and label contract

The source is the UCI Smartphone-Based Recognition of Human Activities and Postural Transitions dataset [1], sampled at 50 Hz with three accelerometer and three gyroscope channels. Windows are 128 samples with stride 64; each window is tied to its recording and raw annotation coordinates. Subject-disjoint splits are fixed as 17 training, 4 validation, and 9 test subjects. Channel order is $[a_x, a_y, a_z, g_x, g_y, g_z]$: raw accelerometer channels are measured in g and gyroscope channels in radians per second. Windows are 2.56 seconds long and advance by a 1.28-second step. Windows never cross recordings. Normalizer statistics are fit across all supervised TRAIN windows and recordings; validation and test never enter the fit. The inputs are raw sensor signals, not the UCI 561-feature representation.

The split audit contains 9,978 full train-stream windows, 2,422 validation windows, and 5,053 test-stream windows. The supervised pure-or-dominant-at-least-0.6 policy contains 6,955 train, 1,751 validation, and 3,572 test windows. The pure-basic total is 10,017 across the corpus; these are different denominators. These counts are audit inputs and must be reproduced from the run manifest before final metrics. Pure windows and majority windows are distinct. Ambiguous or unlabeled stream windows remain in full-stream denominators and are excluded from supervised classification only under the declared policy. The annotation coordinate convention and the mapping from samples to windows must be recorded with each run.

The transition snapshot has 344 pure transition windows and 10,017 pure basic windows among 10,361 pure windows. Existing support gates therefore remain failed for a high-

quality twelve-class training claim. The explicit current decision is to permit an exploratory K=12 feasibility reference with the pure-or-dominant policy, while retaining the status exploratory and making no claim that the historical G2 quality gate passed.

1.3 Frozen reference architecture

The K=12 contract is: input $(B, 6, 128)$; same-padded Conv1D 6→32, kernel 5, padding 2, followed by BatchNorm, ReLU and MaxPool2; same-padded Conv1D 32→64, kernel 3, padding 1, followed by BatchNorm, ReLU and MaxPool2; same-padded Conv1D 64→64, kernel 3, padding 1, followed by BatchNorm and ReLU; Flatten of 2,048 features; Linear 2,048→128, ReLU; and Linear 128→12. The expected trainable parameter count is 283,692 and must be checked from the instantiated K=12 model before training. BatchNorm evaluation uses frozen running statistics; a checkpoint and its preprocessing parameters are an inseparable reference identity.

The operational checkpoint is created only after new training and evaluation. It uses seeds 42, 43, and 44; at most 30 epochs; Adam with learning rate 10^{-3} and weight decay 10^{-4}; cosine scheduling; and patience 10. Epoch and seed selection use validation macro-F1. Test evaluation occurs after selection. Required outputs include accuracy against ground truth, macro-F1, a 12-class confusion matrix, per-class recall, reference-decision agreement, point-margin distributions, and timing by declared scope. These outputs are recorded for the completed feasibility run. Adjacent prediction stability is measured for the snapshot, while ECDR/reference agreement is not established; certificate and full ECDR cost outputs remain pending.

1.4 Research questions and hypotheses

RQ1 (reference quality) What classification accuracy and per-class recall does the new K=12 reference achieve against declared ground truth?

RQ2 (decision preservation) Does ECDR preserve the reference decision with zero disagreements on stable and transition windows under the specified certificate and lifecycle rules?

RQ3 (coverage and cost) At what event-conditioned coverage does ECDR reduce total end-to-end cost relative to evaluating the reference on every window, and how does that compare with a fixed fair-cost policy?

RQ4 (soundness) Which certificate bounds, membership predicates, and lifecycle obligations are executable on this graph?

The hypothesis ledger is conditional. H1 predicts that stable windows will have more stable adjacent reference predictions than transition windows; this is not ECDR/reference agreement; the current snapshot measures it descriptively from 2,560/2,617 stable pairs versus 23/44 transition pairs. H2 predicts that event-conditioned certified coverage can improve at a fixed matched total cost; this is proposed and requires a fair end-to-end benchmark. H3 is a derived theorem obligation: if, for a candidate region $\mathcal{S}_r$ and predicted class c, every competing margin satisfies $\inf_{X \in \mathcal{S}_r}(F_c(X) - F_j(X)) > 0$ for $j \neq c$, then $h(X) = c$ throughout that region. This implication is mathematical, not a validated runtime result; executable bound, arithmetic, membership, and lifecycle checks remain pending. H4 predicts net savings only in an operating regime whose complete cost is measured.

1.5 Domain, event, and certificate semantics

An anchor is a specific window X^r actually evaluated by the frozen reference, with trusted class $c = h(X^r)$. A candidate region is a set of admissible windows, a certificate is a sound lower-bound statement over that region, and membership is the runtime predicate that decides whether a new window belongs to it. These are separate objects. Event descriptors are candidate signals for choosing when to rebuild or test a certificate; they do not prove coverage by smoothness. The whole-domain condition above is a derived condition over all 11 competing margins, not evidence that the condition holds for the current checkpoint. Reference weights, BatchNorm running statistics, preprocessing, tie-breaking, and arithmetic mode must remain fixed across a certificate's lifecycle. Cached hits do not establish a new trusted anchor: reuse is allowed only for a valid certificate and membership result, otherwise the system falls back to the frozen reference.

1.6 Reference classes and architecture audit

The class IDs are fixed as follows: 1 WALKING, 2 WALKING_UPSTAIRS, 3 WALKING_DOWNSTAIRS, 4 SITTING, 5 STANDING, 6 LAYING, 7 STAND_TO_SIT, 8 SIT_TO_STAND, 9 SIT_TO_LIE, 10 LIE_TO_SIT, 11 STAND_TO_LIE, and 12 LIE_TO_STAND. The order is part of the checkpoint identity.

Table 1: Frozen K=12 architecture audit.

Layer	Input	Output	Trainable parameters	Role
Input	$B \times 6 \times 128$	$B \times 6 \times 128$	0	Raw six-channel window
Conv1 (same, k5,p2)	6×128	32×128	992	Local temporal features
BN1+ReLU+Pool2	32×128	32×64	64	Frozen eval affine and downsample
Conv2 (same, k3,p1)	32×64	64×64	6,208	Higher-level features
BN2+ReLU+Pool2	64×64	64×32	128	Frozen eval affine and downsample
Conv3 (same, k3,p1)	64×32	64×32	12,352	Final temporal features
BN3+ReLU	64×32	64×32	128	Fixed evaluation normalization
Flatten	64×32	2,048	0	Preserve positional features
Linear+ReLU	2,048	128	262,272	Dense decision representation
Linear head	128	12	1,548	Twelve logits
Total			283,692	Executed trainable count; shape and parameter checks passed

2 Feasibility baseline results

The full pipeline completed with exit code 0 and 17 internal verification checks. Validation selected seed 43 at epoch 8 by macro-F1 (0.8467853). The selected checkpoint is `selected_best.pt`, SHA-256 `8ffc5dd8eb0e9cdf832160d074f3ae816ae6cba02adf7214a63a8b623303adb1`; the train-only preprocessing identity is `aa8e1cc20829307187e0a78200f9957f7d`

`d068b1dc26063c2720ad832bb2fd6e`. The three seeds selected epochs 14, 8, and 9 for seeds 42, 43, and 44; early stopping means the histories must not be described as 30 completed epochs.

Table 2: Per-seed selection and test variability.

Seed	History length	Best epoch	Val macro-F1	Test accuracy	Test macro-F1
42	24	14	.836829	.886618	.767022
43	18	8	.846785	.890538	.776265
44	19	9	.836958	.889698	.763648

On the supervised test denominator of 3,572, selected seed 43 achieved accuracy 0.8905375 and macro-F1 0.7762645. Across seeds, accuracy was 0.8889511 $\pm$ 0.0016849 and macro-F1 was 0.7689781 $\pm$ 0.0053332, where the spreads are population standard deviations across three seeds, not confidence intervals. The full test stream contains 5,053 windows.

Table 3: Pure-plus-majority support by split; each cell is windows/subjects.

ID	Class	Train	Val	Test
1	WALKING	1077/17	246/4	531/9
2	WALKING_UPSTAIRS	973/17	247/4	537/9
3	WALKING_DOWNSTAIRS	894/17	231/4	492/9
4	SITTING	1103/17	289/4	546/9
5	STANDING	1206/17	312/4	597/9
6	LAYING	1188/17	316/4	586/9
7	STAND_TO_SIT	79/17	19/4	45/9
8	SIT_TO_STAND	53/17	13/4	32/9
9	SIT_TO_LIE	90/17	26/4	51/9
10	LIE_TO_SIT	89/17	15/3	46/9
11	STAND_TO_LIE	119/17	18/3	68/9
12	LIE_TO_STAND	84/17	19/4	41/9

Table 4: Selected-seed class support, recall and F1 (supervised test, $n = 3572$).

ID	Class	Support	Recall	F1
1	WALKING	531	.9567	.9585
2	WALKING_UPSTAIRS	537	.9088	.9208
3	WALKING_DOWNSTAIRS	492	.9837	.9528
4	SITTING	546	.8407	.8196
5	STANDING	597	.8090	.8429
6	LAYING	586	.9949	.9890
7	STAND_TO_SIT	45	.7111	.7356
8	SIT_TO_STAND	32	.6563	.7119
9	SIT_TO_LIE	51	.6471	.6168
10	LIE_TO_SIT	46	.5870	.6667
11	STAND_TO_LIE	68	.5294	.5496
12	LIE_TO_STAND	41	.6585	.5510

The selected-seed confusion matrix uses row and column IDs 1,2,3,4,5,6,7,8,9, 10,11,12 in

the exact class order above; rows are true classes and columns are predicted classes:

	1	2	3	4	5	6	7	8	9	10	11	12
508	0	22	0	0	0	0	0	0	0	0	1	
16	488	15	0	0	0	2	0	0	3	2	11	
4	2	484	0	0	0	2	0	0	0	0	0	
0	21	0	459	57	0	1	2	1	0	5	0	
0	3	0	106	483	0	1	0	1	0	2	1	
0	0	0	0	0	583	0	0	0	0	3	0	
1	2	0	4	1	0	32	3	1	0	1	0	
0	2	0	1	7	0	1	21	0	0	0	0	
0	0	0	0	1	6	0	0	33	0	11	0	
0	1	0	2	0	0	0	0	0	27	1	15	
0	1	1	2	0	4	2	0	20	0	36	2	
0	3	2	0	0	0	1	1	0	5	2	27	

Adjacent prediction stability is descriptive only: among 5,034 valid adjacent pairs (18 recording-boundary/gap pairs excluded), pure stable pairs agreed 2,560/2,617 (97.8219%), pure transition pairs 23/44 (52.2727%), and mixed other pairs 1,399/2,373 (58.9549%). This supports H1 descriptively for this snapshot, but transition support is small and this is not ECDR/reference certification or a population-level independent test.

CPU timing used one thread, FP32, batch shape $1 \times 6 \times 128$, 100 warmup calls and 1,000 varied-input timed calls. Forward-only mean/p50/p95 were 0.173376/0.165034/0.215291 ms; reference-core preprocessing plus forward mean/p50/p95 were 0.201103/0.183905/0.287375 ms. Both scopes exclude raw acquisition, windowing, events, certificate construction, membership, cache overhead and disk I/O. No speedup or net-cost claim follows. The winning point margin median was 5.9782 and the supervised ground-truth margin median 7.5746; these are empirical point margins, not certified bounds. The lowest selected transition recall was STAND_TO_LIE, 36/68 = 0.5294.

No ECDR runtime certificate agreement has been computed. Independent verifier evidence and full net-cost evaluation remain later work; the G2 quality gate is therefore HOLD even though the Week 1 feasibility baseline is complete.

The historical parameterized `SmallCNN1D` implementation is reused to instantiate K=12 when supplied the new class count. Its K=6 configuration and old checkpoints remain historical and cannot transfer metrics or certificates. BatchNorm statistics are fixed in evaluation mode; the official PyTorch 2.14 BatchNorm contract is cited in the bibliography [9].

2.1 Objectives, split subjects, and pipeline

The objectives are separate: (O1) measure classification accuracy and per-class recall against ground truth; (O2) measure reference-decision agreement with a strict zero-disagreement target; and (O3) measure net end-to-end cost against a full-reference baseline. A single score cannot substitute for these three.

The split audit artifact `data/split_audit.json` lists train subjects 1,3,5,6,7,8,11,14,15,16,17,19,21,22,23,25,26, validation subjects 27,28,29,30, and test subjects 2,4,9,10,12,13,18,20,24; it reports subject-disjoint and recording-disjoint checks with zero issues.

The reproducible pipeline is: (1) load raw HAPT recordings and annotations; (2) construct recording-local 128-sample windows at stride 64; (3) attach raw annotation spans and apply the declared pure/dominant label rule; (4) join the fixed subject split; (5) fit channel means

and standard deviations across all supervised training windows only, then reuse them for validation and test; (6) instantiate and count the K=12 graph; (7) train three seeds with the declared protocol and select by validation macro-F1; (8) evaluate test after selection; (9) run agreement, certificate and cost experiments separately. The normalization fit spans training windows across records; windows themselves never cross recording boundaries.

The sole pipeline entry point is `scripts/week1_reference_pipeline.py`. The help-verified all-run command is `./.venv/bin/python scripts/week1_reference_pipeline.py all --output-dir output/week1_closeout_20261102/reference --epochs 30 --seeds 42,43,44 --device cpu`. No legacy K=6 command or nonexistent K=12 config is presented as a reproduction command. A fresh reproduction must substitute a new output directory; the recorded directory is immutable evidence. The run record must preserve run ID, command, environment, checkpoint hash, model config hash, preprocessing hash, data manifest hash, and seed.

The current audit artifact is run `week1_ref_1793620216_9d6c1d85`, created 2026-11-02T11:50:16Z, with pipeline hash `1ef93101f658f8a18ebf19f0c8743ec5311ffe94933aee3505be6ec29a0be4d6`. It covers 131 source/data hashes, 122 raw sensor files, 17,453 windows from 61 recordings, float32 shape $17{,}453 \times 6 \times 128$, zero raw-slice mismatches, zero label-builder mismatches, and verified aligned overlap for all pairs. A one-sample offset changes 98 window policy/label assignments; this is an explicit provenance caveat and does not change the current run. The CPU environment is Xeon W-11855M, Python 3.11.15, PyTorch 2.14.1+cpu, NumPy 2.4.6, pandas 3.0.6, one thread. These are audit and feasibility results, not a quality-gate pass; ECDR certification and net-cost evaluation remain pending.

Table 5: Feasibility outputs and remaining external dependencies.

Field	Evidence	Status
Per-class support/recall	Selected test table, all 12 classes	VERIFIED RUN
Confusion matrix	12 by 12 with class IDs 1–12	VERIFIED RUN
Adjacent stability	5,034 valid pairs, stable/transition strata	VERIFIED DESCRIPTIVE
Reference timing	Mean/p50/p95 preprocessing plus forward	VERIFIED BASELINE
Three-seed variability	Selected epochs and population spread	VERIFIED RUN
ECDR/reference agreement	Runtime certificate comparison	NOT ESTABLISHED
Certificate and net cost	Independent verifier and full ECDR benchmark	WEEK 3 DEPENDENCY

Table 6: Hypothesis ledger and measurement boundary.

ID	Basis	Test	Status
H1	Stable/transition event distinction	Adjacent reference-prediction stability; 2,560/2,617 versus 23/44, descriptive only	MEASURED DESCRIPTIVE
H2	Event-conditioned reuse may avoid unnecessary rebuilds	Coverage versus fixed fair-cost curve	PROPOSED
H3	All 11 positive regional margins imply one class	Sound bound, membership and lifecycle obligations	DERIVED
H4	Reuse can save net cost in a workload regime	Full end-to-end timing including fallback and overhead	PROPOSED

Table 7: Native Week 1 evidence register.

ID	Scope and artifact	Status
W1-01	Research grounding and targeted primary literature	VERIFIED
W1-02	K=12 lock, class order and frozen checkpoint contract	VERIFIED RUN
W1-03	Exact architecture graph and 283,692 trainable parameters	VERIFIED RUN
W1-04	Executed full reference pipeline and reproducibility artifacts	VERIFIED RUN
W1-05	Transition label policy and support blocker	VERIFIED SNAPSHOT
W1-06	Main research question and separated objectives	VERIFIED
W1-07	Evidence register with claim status	VERIFIED
W1-08	Sensor/data full contract and denominators	VERIFIED SNAPSHOT
W1-09	Hypothesis ledger H1–H4	PROPOSED/DERIVED
W1-10	Stable/transition stratification plan	VERIFIED DESCRIPTIVE
W1-11	Week roadmap and dependencies	VERIFIED
W1-12	English presentation and editable artifacts	VERIFIED

2.2 Certificate lifecycle and architecture-specific firewall

Reuse is permitted only when a valid certificate exists and membership holds. Otherwise the system falls back to the fixed reference evaluation. A hit does not renew the trusted anchor. Model or preprocessing changes invalidate the certificate, as do recording or gap-boundary changes that violate the declared region. The mathematical condition quantifies every point in the candidate region and every one of the 11 competing classes; it does not assume that a stable window is correct, and point margins are not regional bounds.

The exact overlap between adjacent stride-64 windows is 64 samples across six channels, hence 384 shared coordinates. This is a reuse opportunity, not a zero-difference input: the remaining coordinates change, and the full-input distance must be computed under the declared norm if a bound requires it. The old valid-convolution/GAP feature cancellation formulas are therefore firewalled from this BatchNorm–MaxPool–Flatten graph. Week 3 must derive or verify bounds on this actual graph.

2.3 Grounding and research gap

The targeted primary literature covers Lipschitz-margin certification [2], IBP [3], CROWN and auto-LiRPA [4, 5] for sound bounds; StreamiNNC and TinyDejaVu [6, 7] for streaming reuse or sensor-stream efficiency; and LipCache/GuardNet [8] for certified caching boundaries. These works motivate the questions but do not establish a result on this HAPT contract. No novelty, universal speedup, or superiority claim is made here. A GuardNet certificate must not be described as a certificate for a different MainNet without matching the reference boundary. The earlier Conv-valid–ReLU–GAP derivation is architecture-specific and is not transferred to the BatchNorm–MaxPool–Flatten reference.

2.4 Evidence status and roadmap

Evidence is labeled VERIFIED (read from a source artifact and checked), DERIVED (a logical consequence whose premises remain to be checked), PROPOSED (a planned measurement

Table 8: Targeted related-work boundary.

Work	Reference guaranteed	Payload/domain	Cost/evidence boundary
Lipschitz-margin [2]	Perturbation margin under stated norm	Classifier robustness	No HAPT ECDR cost result
IBP/CROWN/auto-LiRPA [3, 4, 5]	Sound bounds when relaxations and arithmetic hold	Neural-network regions	Bound runtime is method/graph dependent
StreamiNNC [6]	Streaming CNN computation reuse	Shifted windows; padding caveats	Efficiency evidence is workload-specific
TinyDejaVu [7]	Sensor-stream inference design	MCU RAM and latency	No transfer of benchmark numbers
LipCache/GuardNet [8]	Certified cache proxy boundary as defined by the work	Edge image service	GuardNet/MainNet scope must not be conflated

or method), or NOT-ESTABLISHED. Week 1 closes the brief, data and architecture scope lock, and a feasibility baseline. Week 2 documents why each data and deep-model component is used. Week 3 checks certificate mathematics and executable bounds on the actual graph. GAP-specific formulas from the old theory and any deep revisions are dependencies for later weeks, not Week 1 results.

3 Source and reproducibility contract

Every reported number must cite a run ID, pipeline script, command, environment and provenance hash. The closeout register records source file, scope, denominator, status, and whether a statement is current, historical, or proposed. The parameterized `scripts/backbone.py` implementation can instantiate the current K=12 graph; the K=6 configuration and old checkpoints are historical evidence only and cannot transfer metrics or certificates. User acceptance is not asserted by this document.

References

[1] UCI Machine Learning Repository, *Smartphone-Based Recognition of Human Activities and Postural Transitions*, 2015. https://archive.ics.uci.edu/dataset/341/smartphone+based+recognition+of+human+activities+and+postural+transitions.

[2] Tsuzuku, Sato, Sugiyama, *Lipschitz-Margin Training*, NeurIPS 2018. https://arxiv.org/abs/1802.04034.

[3] Gowal et al., *On the Effectiveness of Interval Bound Propagation*, 2018. https://arxiv.org/abs/1810.12715.

[4] Zhang et al., *Efficient Neural Network Robustness Certification with General Activation Functions*, NeurIPS 2018. https://arxiv.org/abs/1811.00866.

[5] Xu et al., *Automatic Perturbation Analysis for Scalable Certified Robustness and Beyond*, NeurIPS 2020. https://arxiv.org/abs/2002.12920.

[6] Kechris et al., *Don't Think It Twice*, 2024. https://arxiv.org/abs/2408.03223.

[7] Huang and Baccelli, *TinyDejaVu: Smaller RAM and Faster Inference with Neural Networks on MCUs for Sensor Data Streams*, arXiv v2, 2026. https://arxiv.org/abs/2512.09786v2.

[8] Xiang et al., *LipCache: A Local Inference Proxy with Certified Caching for Edge Image Classification Service*, arXiv v1, 2026. https://arxiv.org/html/2608.13144v1.

[9] PyTorch, *BatchNorm1d*, documentation for the 2.14 API, accessed 2026-11-02. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm1d.html.